Demonstrating execl() in C

Using fork(), process IDs, waitpid(), and exit()

1. Complete C program

The parent creates a child with **fork()**. The child calls **execl()** to replace its program with the Linux **ls -l** command. The parent waits for the child and reads its exit status.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid;
    int status;

    printf("Parent process started\n");
    printf("Parent PID = %ld\n", (long)getpid());
    fflush(stdout);

    // Create a child process
    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(EXIT_FAILURE);
    }
    else if (pid == 0)
    {
        // Child process
        printf("\nChild process started\n");
        printf("Child PID = %ld\n", (long)getpid());
        printf("Child is executing ls -l using execl()\n\n");
        fflush(stdout);

        // Replace the child process with the ls command
        execl("/bin/ls", "ls", "-l", (char *)NULL);

        // This executes only if execl() fails
        perror("execl failed");
        exit(EXIT_FAILURE);
    }
    else
    {
        // Parent process
        printf("\nParent is waiting for child PID = %ld\n",
               (long)pid);

        if (waitpid(pid, &status, 0) == -1)
        {
            perror("waitpid failed");
            exit(EXIT_FAILURE);
        }

        printf("\nChild process has finished\n");

        if (WIFEXITED(status))
        {
            printf("Child exit status = %d\n",
                   WEXITSTATUS(status));
        }
        else
        {
            printf("Child did not exit normally\n");
        }

        printf("Parent process continues\n");
    }

    return 0;
}
```

2. What execl() does

execl("/bin/ls", "ls", "-l", (char *)NULL)

• **/bin/ls** — path to the executable.
• **ls** — argument 0, conventionally the program name.
• **-l** — command-line option passed to ls.
• **(char *)NULL** — terminates the argument list.

Key point: execl() does not create a new process. It replaces the current process's program. The child retains the same PID, but its program image is replaced.

3. What happens after execl()?

If execl() succeeds, it does not return to the original C code; the ls program runs instead. If it fails, execl() returns -1, so perror() reports the error and the child exits with EXIT_FAILURE. The parent can still collect the child's status using waitpid().

4. Process flow

```
Parent starts
    |
  fork()
    +--------------------------+
    |                          |
    v                          v
Parent receives child PID    Child receives 0
    |                          |
waitpid(child PID, ...)     execl("/bin/ls", ...)
    |                          |
wait for child              ls -l runs in child
    |                          |
read exit status <-------------+
    |
Parent continues
```

5. Compile and run

```
gcc fork_execl_demo.c -o fork_execl_demo
./fork_execl_demo
```

Save the code as **fork_execl_demo.c**. Run it in Linux or a POSIX-compatible environment such as WSL. The listed files and process IDs will depend on your system.

6. Quick knowledge check

Question: If execl() succeeds, will the next line in the original child code execute?
Answer: No. Successful exec replaces the process image. The next line runs only if execl() fails.

Experiment: Change **"-l"** to **"-a"** in the execl() call and observe how the directory listing changes.